

TEMPLATE DE DOCUMENTAÇÃO DE DECISÕES - PBL

Disciplina: Manutenção de Software e DevOps

Professor: Daniel Souza

Dados da equipe

- **PBL Selecionado:** Caso 1 — Chamados de Laboratório

- **Link do projeto):** <https://github.com/MarceloWilson/DevOPS-PBL>

- **Turma:** OADSNM2BB

- **Integrantes:**

- 24114290055 - Marcelo Wilson de Santana - Líder
- 24114290034 - Vinicius Teixeira Campos
- 24114290133 - Matheus Lima Rodrigues
- 24114290095 - Kauã Henrique Santos de Araújo
- 24114290091 - Ana Laura castro do Nascimento

1. Contexto do problema

1.1 Descreva o problema:

Atualmente, os chamados de laboratório são registrados por mensagens avulsas, dificultando o controle, o acompanhamento e a identificação dos responsáveis pelas solicitações. A falta de um sistema centralizado pode causar perda de informações e atrasos no atendimento.

1.2 Objetivo do sistema:

Desenvolver um sistema para gerenciar chamados de laboratório de forma simples e organizada, permitindo cadastrar, listar, alterar status e excluir chamados, facilitando o controle e o acompanhamento das solicitações.

1.3 Escopo mínimo (o que foi implementado):

Foi implementado um sistema básico de gerenciamento de chamados contendo as seguintes funcionalidades:

- Cadastro de chamados;
- Listagem de chamados cadastrados;
- Alteração do status dos chamados (Aberto e Finalizado);
- Exclusão de chamados;
- Integração com banco de dados MySQL 8.0;
- Backend e frontend containerizados com Docker;
- Orquestração dos serviços utilizando Docker Compose;
- Pipeline CI/CD para execução de testes e build da aplicação.

2. Decisões de banco de dados

2.1 Entidades principais do problema:

A principal entidade do sistema é **Chamado**, responsável por armazenar as informações de cada solicitação registrada no laboratório.

Chamado

- ID
- Título
- Descrição
- Responsável
- Prioridade
- Status
- Data de Criação ([createdAt](#))

2.2 Modelo descritivo:

Todo chamado possui um identificador único (ID), título, descrição, responsável, prioridade, status e data de criação.

- Cada chamado é registrado por um responsável.
- Cada chamado possui apenas um status, que pode ser **ABERTO** ou **FINALIZADO**.
- Cada chamado possui uma prioridade para indicar seu nível de importância.
- O sistema permite cadastrar, listar, alterar o status e excluir chamados.
- Todos os chamados são armazenados em banco de dados para possibilitar o controle e acompanhamento das solicitações realizadas no laboratório.

2.3 Diagrama Entidade-Relacionamento:

Diagrama Entidade-Relacionamento (DER)

CHAMADO		
 PK	id	INT
—	titulo	VARCHAR(255)
—	descricao	TEXT
—	responsavel	VARCHAR(255)
—	prioridade	VARCHAR(50)
—	status	VARCHAR(50)
—	createdAt	DATETIME

Legenda

-  PK Chave Primária
- Atributo

2.4 Diagrama Relacional:

Diagrama Relacional

(Modelo Relacional)

CHAMADO	
 id (PK)	INT
titulo	VARCHAR(255)
descricao	TEXT
responsavel	VARCHAR(255)
prioridade	VARCHAR(50)
status	VARCHAR(50)
createdAt	TIMESTAMP

Legenda:

-  PK = Chave Primária
- VARCHAR = Texto de tamanho variável

2.5 Modelo Físico (SQL - DDL):

```
CREATE DATABASE labdesk;  
USE labdesk;
```

```
CREATE TABLE Chamado (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  titulo VARCHAR(255) NOT NULL,  
  descricao TEXT NOT NULL,
```

```
responsavel VARCHAR(255) NOT NULL,  
prioridade VARCHAR(50) NOT NULL,  
status VARCHAR(50) NOT NULL DEFAULT 'ABERTO',  
createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3. Decisões de backend

3.1 Estrutura da API (rotas principais):		
---	--	--

Método	Rota	Descrição
GET	/api/chamados	Lista todos os chamados cadastrados
POST	/api/chamados	Cria um novo chamado
PATCH	/api/chamados/:id/status	Atualiza o status de um chamado
DELETE	/api/chamados/:id	Remove um chamado do sistema

3.4 Alternativas consideradas e decisão final:		
---	--	--

Foram analisadas opções para o desenvolvimento da aplicação e, ao final, foram escolhidos **React, Node.js, MySQL e Prisma** por sua simplicidade, integração e facilidade de utilização. Além disso, foram utilizados **Docker, Docker Compose e GitHub Actions** para atender aos requisitos DevOps do projeto.

4. Decisões de infraestrutura com Docker

4.1 Serviços definidos no docker-compose:
--

Durante o desenvolvimento do projeto, foram avaliadas diferentes opções para construção da API e do banco de dados. A equipe optou por utilizar **Node.js com Express** no backend devido à sua simplicidade, rapidez de desenvolvimento e boa integração com aplicações web.

Para o acesso ao banco de dados foi escolhido o **Prisma ORM**, que facilita a manipulação dos dados e reduz a complexidade das consultas SQL.

Como banco de dados, foi utilizado o **MySQL 8.0**, por ser uma solução robusta, amplamente utilizada e compatível com as necessidades do sistema.

No frontend, foi adotado o **React**, permitindo a criação de uma interface moderna e intuitiva para gerenciamento dos chamados.

Por fim, foram utilizados **Docker**, **Docker Compose** e **GitHub Actions** para atender aos requisitos DevOps, garantindo padronização do ambiente, automação de testes e integração contínua da aplicação.

4.2 Variáveis de ambiente utilizadas em cada serviço:

O sistema utiliza variáveis de ambiente para configurar a aplicação e a conexão com o banco de dados.

Backend (Node.js):

```
DATABASE_URL="mysql://root:iesb@mysql:3306/chamadosdb"
PORT=3000
```

Banco de Dados (MySQL):

```
MYSQL_ROOT_PASSWORD=iesb
MYSQL_DATABASE=chamadosdb
```

Frontend (React):

```
VITE_API_URL=http://localhost:3000/api
```

Essas variáveis permitem configurar a aplicação sem a necessidade de alterar o código-fonte, facilitando a execução em diferentes ambientes e containers Docker.

4.3 Persistência de dados (volumes):

Foi utilizado um volume Docker para armazenar os dados do banco MySQL de forma permanente. Assim, as informações dos chamados permanecem salvas mesmo que os containers sejam reiniciados ou recriados. Isso garante maior segurança e integridade dos dados da aplicação.

4.4 Rede e comunicação entre serviços:

A comunicação entre os serviços é realizada por meio de uma rede criada pelo Docker Compose. Dessa forma, o frontend, o backend e o banco de dados MySQL conseguem se comunicar de forma segura e integrada dentro do mesmo ambiente, utilizando os nomes dos serviços definidos no arquivo docker-compose.yml. Isso facilita a integração entre os componentes da aplicação e simplifica a configuração do ambiente.

4.5 Dificuldades encontradas e solução adotada (opcional):